From a Lean source to a reviewable explanation

An architectural companion to Lean Exposition

23 October 2026 · first draft, not reviewed

Draft scope. This companion explains the repository's present file and command contracts. It has not received independent mathematical or reader review. The repository's QAOA/SK papers are a worked example, not a claim that the system has verified their mathematical interpretation.

1. Why keep more than a proof file?

A Lean theorem answers a precise question: whether a term has the type stated in a particular formal environment. A mathematician reading a paper asks additional questions. Are the hypotheses the ones the prose describes? Which source proves the analytic step that Lean does not contain? Does the short account omit a condition retained in the long one? The answers can change while the Lean proof still compiles.

Lean Exposition keeps these questions connected without making them identical. Its local evidence graph records selected declarations, assumptions, cited sources, observations and the relations among them. Three authored texts then serve different readers: a long reasoning record, a shorter mathematical paper, and an expository account. A mapping alongside each text binds passages to the exact graph edition and records the state of review. The repository packages these materials for inspection; it does not generate a proof from prose or certify that the prose means what the author intended.

The SK example makes the distinction concrete. OpenAI's full-support manuscript argues that an admissible minimiser of the zero-temperature Parisi functional has full relative Stieltjes support on $[0, 1)$. A separate scalar-value argument and a further QAOA construction use related mathematics but do not follow merely from the words "full support." The graph can record those distinct claims and their sources. The papers must still explain the analytic bridges and say which steps are formalised, source-checked, conditional or open. The current drafts retain unresolved review obligations; no bundle status turns them into accepted theorems of exposition.

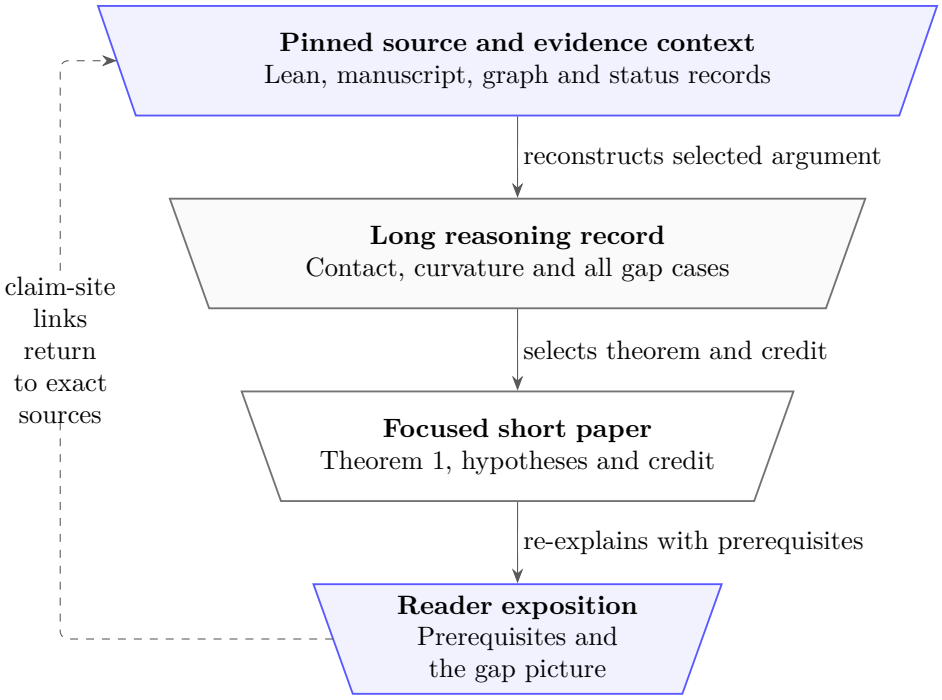


Figure 1: An inverted reader funnel adapted from Will Cook's existing mathematics-map visual. It selects a way into one scoped result; the widths represent editorial selection, not page counts, proof strength or the length of the expository paper. Solid arrows name authoring transitions. The separate dashed route returns a reader from a claim to pinned evidence, without asserting that any text or check certifies mathematical meaning.

2. One route from source to reader

The starting material is a chosen, pinned source: for example, a Lean checkout with its toolchain and a mathematical manuscript with a known edition. An author or extractor records relevant declarations and evidence in a graph. A theorem that uses a lemma points to that lemma in a dependency relation. A claim's link to a selected formal statement is a different, formalisation relation; a citation identifies an external source edition. The reader can follow the recorded relation from a claim to its stated support, or ask which other claims refer to a changed item. The meaning and direction of each link remain visible.

An author can work through the SK example in a deliberate order. The long account reconstructs the full support-gap calculation and its dependence on the variation formula. The short paper distils the mathematical argument with its hypotheses visible. The expository account explains why a hypothetical gap conflicts with the moment identity before returning to the formal statement. These are authored stages, not automatically generated summaries. Their emphases differ, but a passage map binds each account to the same selected graph edition. If an author changes the source graph or a passage, the corresponding byte hash changes and assembly requires a revised mapping. A successful hash check establishes correspondence to recorded bytes, not that the explanation is true.

The graph supports several *typed views* of the same records. A declaration view follows formal Lean dependencies; an argument view shows curated support and formalisation relations with their own meanings and directions; a citation view follows sources; a platform view follows public

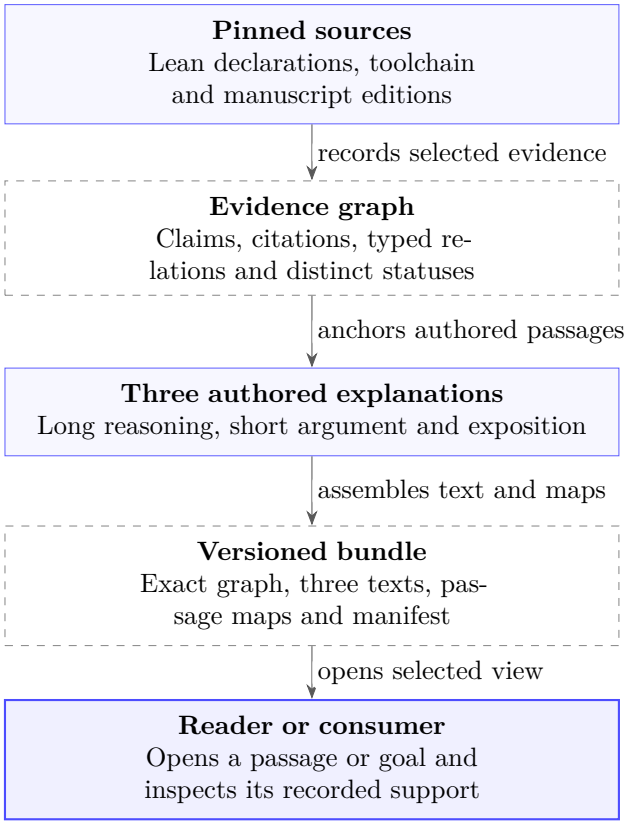
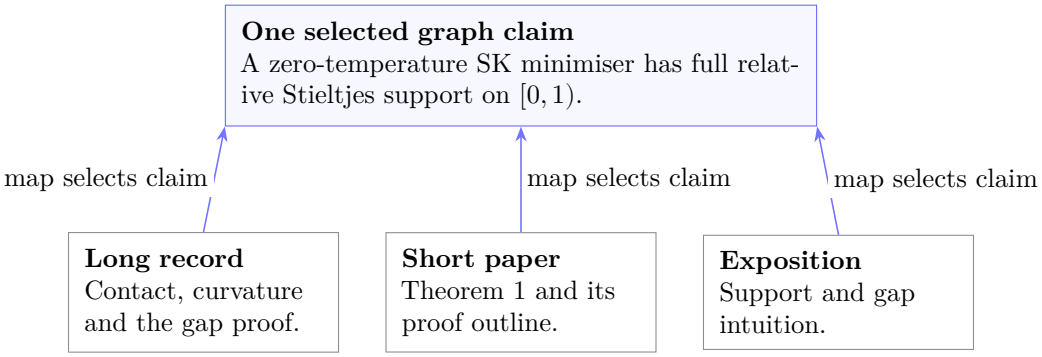


Figure 2: Production and inspection workflow. The bundle includes the exact graph edition as well as authored papers and maps; its byte checks do not judge mathematical meaning. Adapted from the authored-records and derived-views composition in the Plectis Erdős systems paper.



Each passage has its own text and mapping. These links select the same claim; they do not make the accounts interchangeable or reviewed.

Figure 3: Passage-to-claim correspondence, not a proof dependency. Three authored passages can point to one recorded mathematical claim. The forked composition adapts the shared-support, three-version figure in the Plectis Erdős systems paper.

decomposition metadata; and a provenance view follows recorded source and receipt links. A selected goal determines where traversal starts, without removing unrelated evidence from the underlying graph. A view of no recorded links reports an absence in this graph, not mathematical independence.

A reader may instead start from one paper or a named passage in it. The paper projection returns its section outline and mapped passages in actual reading order, with the text and exact byte and line spans. It links each passage to the claims, citations and assumptions recorded in its map, and then shows their prerequisite support subgraph with the original Lean links, evidence and statuses. This reading-order relation is different from the mathematical dependency relation. The projection also reports unmapped text and any passages excluded by a selector, so a narrow view cannot masquerade as a complete reading. Related views retain references to the same graph edition across all three papers; no new graph or bundle member is created.

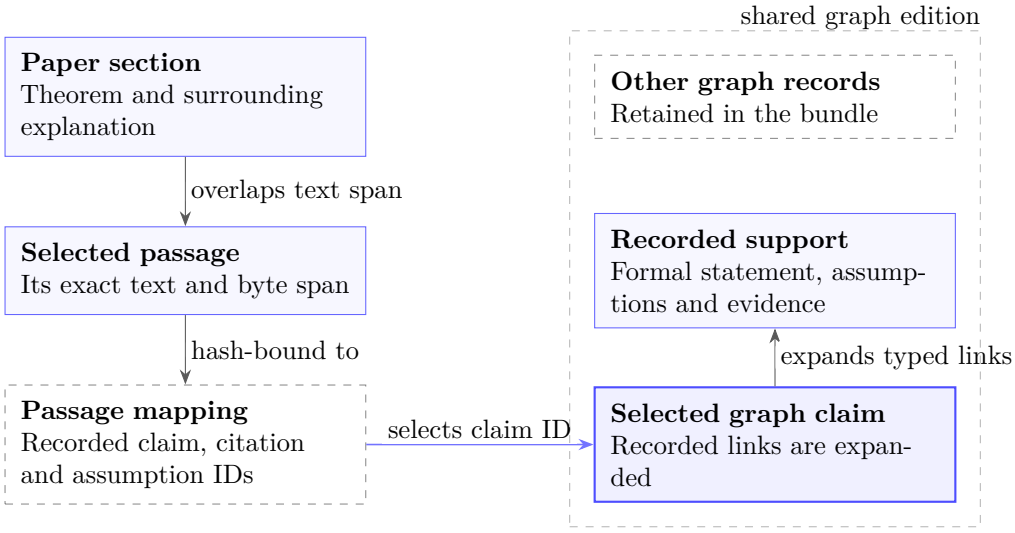


Figure 4: A local paper view follows document structure, passage-to-claim mapping and recorded argument relations without discarding other graph records. The selected support includes typed relations whose meanings and directions remain explicit; this is not a uniform proof graph. The selection-and-support shape adapts the registered-support figure in the Plectis Erdős systems paper.

Cycles may occur among recorded relations. The consumer reports their strongly connected components, the kinds of links within and between them, and a directed acyclic graph of the components. This is a way to navigate mutually connected records; it is not a proof that a circular mathematical argument is valid. Choosing the reverse "consumers" direction can show what refers to a declaration. Neither direction silently changes the original evidence or its status.

3. What each check can say

Lean, Comparator and Prove2Me answer different questions. A successful Lean build concerns a particular formal statement in a pinned environment. The Comparator runs a separately stated challenge against a selected implementation and axiom policy. A platform observation records public metadata at a time: visibility, status, environment and decomposition links. These observations must stay separate. In particular, a platform's "proved" label does not mean this repository replayed the proof, and a Comparator startup failure is not a mathematical counterexample. The QAOA/SK research target has a retained historical unavailable Comparator startup. The evidence

bound to the current papers contains no terminal research-target replay receipt; this companion does not upgrade that history to a pass.

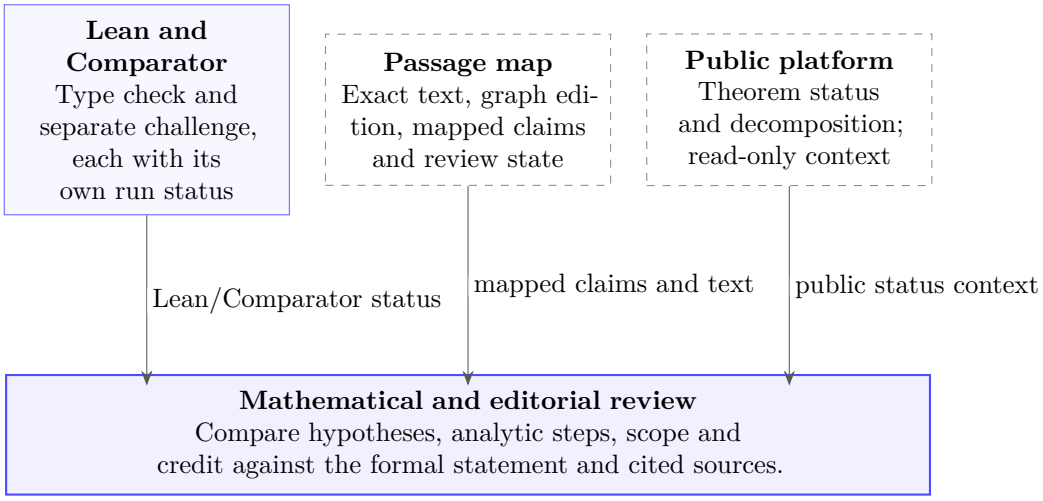


Figure 5: Three distinct records can inform a later human review. Their arrows specify what each channel supplies; none certifies mathematical interpretation or says review has occurred. Adapted from the two-correspondence figure in the Plectis Erdős systems paper.

The system deliberately reports `not_run`, `unverified`, `not_reviewed` and `needs_revision` where they apply. A build, a source hash, a graph closure and a rendered PDF each establish a narrower fact than human comprehension. One can inspect the records now while still asking a mathematician to check the argument.

4. Why the bundle has eight files

The portable bundle has a manifest, the graph, and one Markdown text with a passage map for each of the three stages. The manifest records the format, local publication state and hashes of the other seven files. The graph keeps typed evidence and status. Each map binds its authored text to exact graph bytes, selected goals, claims, citations, assumptions, passages and review state. The pairings keep mathematical prose readable while making its recorded support inspectable.

Removing a map would hide one account's graph correspondence; merging the texts would erase their distinct audiences. Lean source, manuscript editions and verification receipts remain external and are identified by records and digests where available. A consumer validates the directory or ZIP and reads selected evidence or paper views. The bundle is not a self-contained proof archive, and its `semantic_certification` field is false.

The bundle name and format version do not identify an immutable reviewed edition. Keep the manifest's SHA-256 with the review decision and source revisions. A changed passage or graph needs a new edition and review of the affected claims; an approval cannot drift onto later bytes. The mappings and receipts preserve this distinction without asking every reader to open JSON.

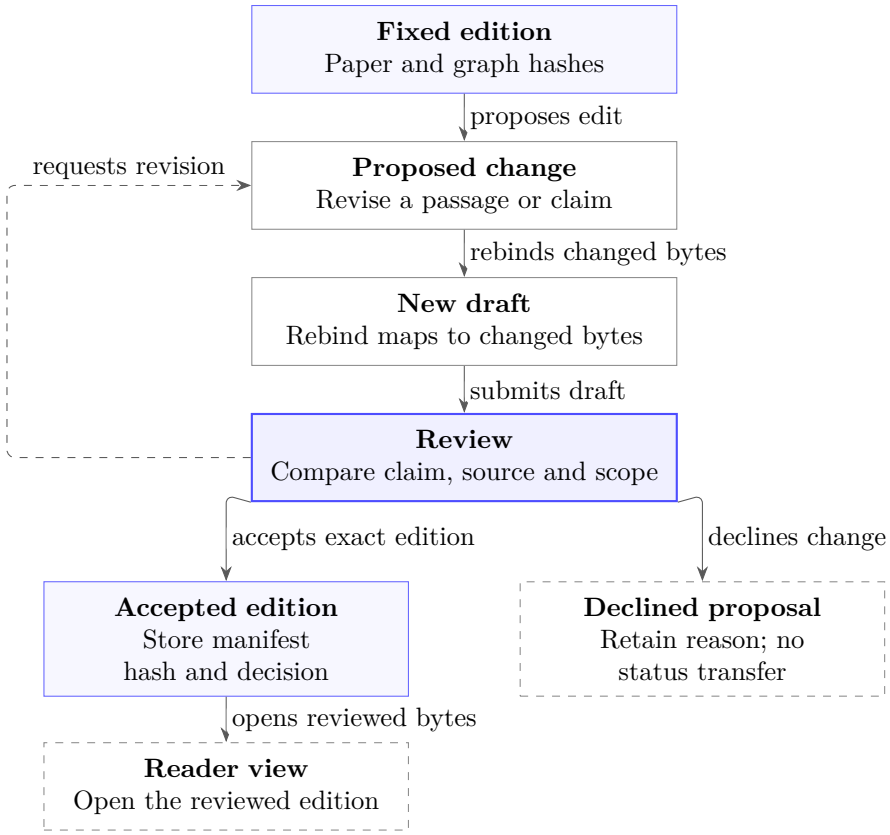


Figure 6: Review-state transitions for an immutable edition. A revision creates new bytes and does not inherit an earlier decision. The accept/revise/decline composition adapts the refinement figure in the Plectis Erdős systems paper. The present QAOA/SK drafts have not been approved.

The reading experience can be simpler. A PDF can lead with the argument and offer source and status links at the relevant claim. A consumer can show a small paper or theorem view, expand dependencies on request, and leave hashes in the manifest. A duplicate derived view can be regenerated from the graph. These changes must preserve source versus interpretation, formal check versus review, and one edition versus another.

5. Use and external handoff

The local command-line tool validates graphs, selects goals, creates empty scaffolds, assembles authored paper-and-map pairs, and checks or inspects a bundle by graph, paper or passage. Assembly requires valid existing hashes and schema; it does not write mathematics. The repository README gives the commands, and the QAOA/SK example supplies a source route.

The optional Prove2Me adapter reads partial public theorem and decomposition metadata using an existing credential and pinned documentation version. It filters private or unknown children; it does not submit proofs, upload papers or receive bundle reviews. The Prove2Me paper separates immutable statements, proof submissions and human audit (Chen, Marwaha, Lu, Yuen and Peng, 2026, §§3–4). This is an analogy, not evidence these papers were attached to a theorem card. An external paper handoff would need an explicit edition, attachment and feedback contract.

This companion is a first, unreviewed explanation of the current local design. The next review

should test whether a mathematician can start at a paper claim, find the exact formal and analytic sources, and understand which verification steps are complete without learning the internal file layout first.

Sources. This draft follows the local CLI, graph, bundle and consumer contracts and QAOA/SK example as observed on 23 October 2026. Figure 1 adapts Will Cook's existing mathematics-map video funnel. Figures 2–6 adapt Will Cook's *A Repository-Based System for Research and Publication* (CC BY 4.0). The Prove2Me boundary follows Chen, Marwaha, Lu, Yuen and Peng, *Prove2Me*, §§3–5; its case studies do not evaluate Lean Exposition.